

Electronic Design Automation (EDA)

EDA Ecosystem

Includes EDA software, tools, and methods from specifications to tapeouts

EDA Tool Type	Purpose	Key Methods / Examples
Simulation	Predict circuit behavior before implementation	HDL simulation, emulation, rapid prototyping
Design	Assemble circuit elements and layout geometries	Logic synthesis, place and route, custom layout
Verification	Ensure correctness and performance of design	Physical verification, LVS, functional verification, formal

Digital, analog, RF, mixed-signal, packaging, PCB

Design for manufacturability, reliability, security, ...

Intellectual property (IP) reuse – do not reinvent the wheel!

What is EDA?

EDA is a market segment consisting of software, hardware, and services with the collective goal of assisting in the definition, planning, design, implementation, verification, and subsequent manufacturing of semiconductor devices, or chips.

Source: <https://www.synopsys.com/glossary/what-is-electronic-design-automation.html>

EDA is at the heart of the heart of modern IT – Dr. Aart de Geus (co-founder and executive chair of Synopsys)

No EDA → No chip or semiconductor industry as we know of today!

EDA market size: \$16 billions in 2024

2.5%

Semiconductor industry: \$628 billions in 2024

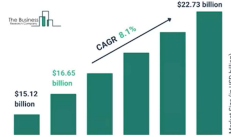
5%

Global IT industry: \$11.7 trillions in 2024

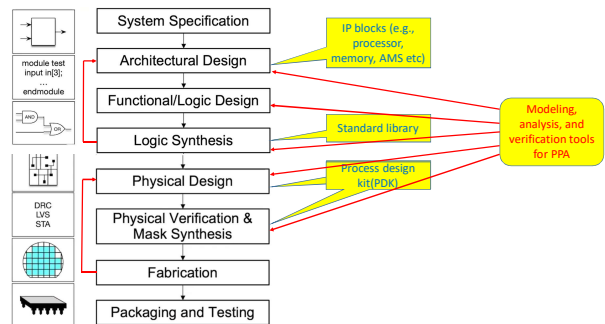
10%

Global GDP: \$110 trillions in 2024

Electronic Design Automation (EDA) Global Market Report 2025



Chip Design Flow



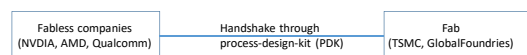
Chip Design

Modern chips are "system-on-chip" – integrating many components to a single chip

- Digital: CPU, GPU, memory, control logic, ...
- Analog: RF, sensors, and mixed-signals

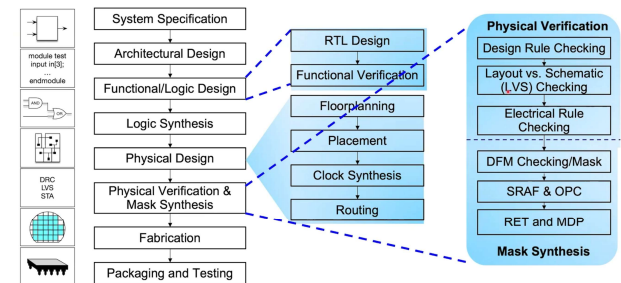
Deals with unbelievable complexities: billions of transistors and 100's km of wiring, in your nail size !

Very expensive to tapeout (e.g., \$50 millions in 5nm) – verify verify!

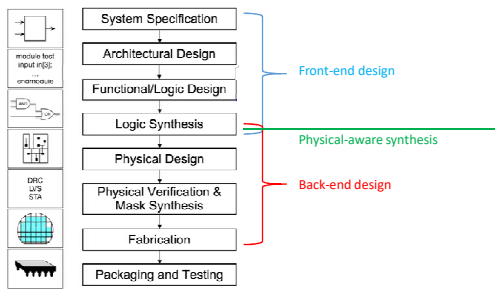


To deal with such extreme complexities → divide and conquer

Chip Design Flow



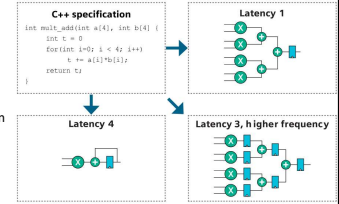
Chip Design Flow



High Level Synthesis (HLS)

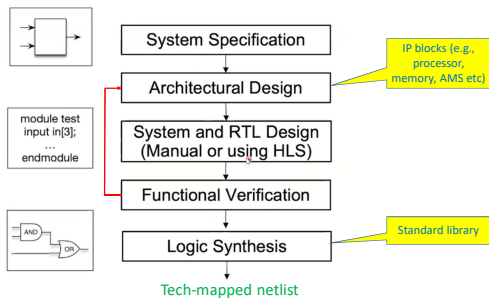
From high-level code (C/C++) to hardware description (Verilog)

- Control and data flow analysis
- Resource allocation
- Scheduling
- Functional unit and register binding
- Resource sharing
- Other optimizations



Large language model (LLM) for RTL generation – rapidly evolving!

Front-end Design



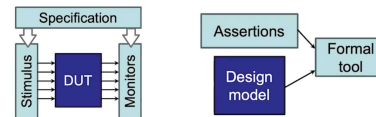
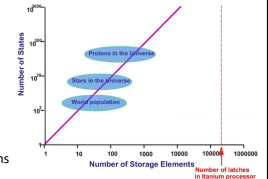
Functional Verification

State space increases exponentially with an increase in the design complexity

Verification is extremely important
✓ Current practice: 1 RTL designer needs 2-3 verification engineers

Two main methods

- Mostly rely on testbench and simulation, but the coverage?
- Formal verification: 100% coverage but works on small designs



Emulations: loads design into emulation hardware and runs for days/weeks

Front-end Design

The main goal of the architecture and system-level design is to generate functionally correct and good RTL/Verilog

- Architecture and system-level simulators
- Make high-level decisions like how many GPU cores, memory size, interconnect architecture etc

Still heavily manual (based on designer experiences, previous designs etc) but growing usage of high-level synthesis (HLS)

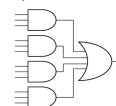
Functional verification

Logic synthesis to generate the netlist, map it to the IP blocks / standard library cells

Technology Independent Logic Synthesis

Two level minimization: sum-of-products (SOPs) and product-of-sums (POS)

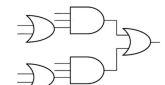
- Smaller delay, more gates
- Minimize the number of minterms
- Example: Quine-McCluskey, ESPRESSO



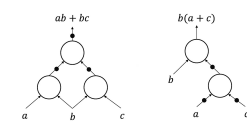
$$F = abc + abd + acd + bcd$$

Multi-level minimization:

- allow more logic sharing
- Larger delay, fewer gates



$$F = ab(c+d) + cd(a+b)$$



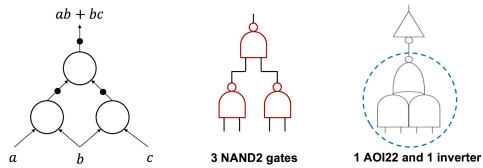
And inverter graph (AIG)

- Boolean network: only AND + NOT gates
- Directed acyclic graph
- Minimize number of nodes (area) and depth (timing)

Technology Mapping

Converts a technology-independent logic network into an optimized gate-level netlist using a predefined library (standard cells/IPS)

Decide proper cell types, sizes, and threshold voltage for PPA optimization



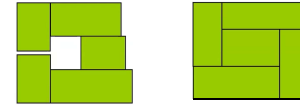
Floorplanning

Each partition can be considered as a macro

- Soft block (flexible shapes)

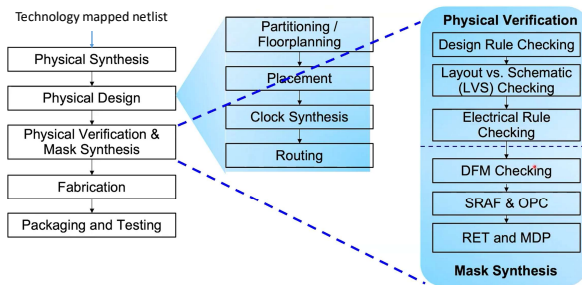
- Hard block (fixed shape but orientation can be changed)

Floorplan all these macros to optimize certain objectives, such as area, wire length, timing



Simulated annealing, reinforcement learning can be used to find a good floorplan

Back-end Design



Placement

Placement decides the exact location of each standard cell or hard macro

Modern designs are huge: 10M+ cells, and hundreds of macros

Floorplanned vs unconstrained placement

A classical NP-Hard problem

- Divide and conquer
- Global placement
- Legalization
- Detailed placement

Nonlinear optimization: ePlace/RePlace

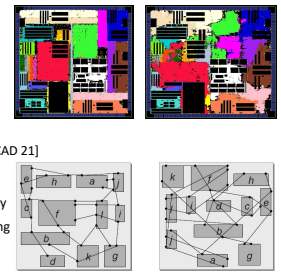
Deep learning + GPU acceleration: DREAMPlace [DAC19, TCAD 21]

Placement to consider timing and routability

Bad placement will have severe consequences on routability

Predicting congestion without performing routing, e.g., using ML or other faster approaches

Then spread cells in congested area

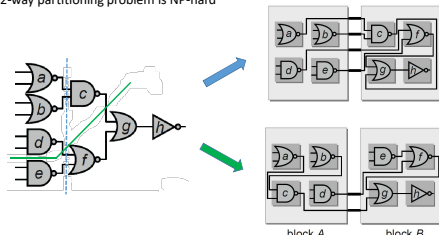


Partitioning

Two-way or multi-way partitioning to divide a big circuit into smaller circuits

Minimize interconnections between different partitions (i.e., total cut size)

A simple 2-way partitioning problem is NP-hard



Clock Network Synthesis

Placement decides the location of flip flops and registers

Modern digital designs are synchronized by clock signals

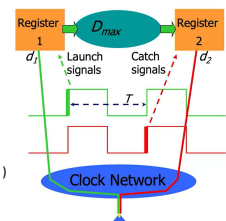
All registers are supplied with clock signal by clock distribution network

Skew = $d_1 - d_2$

Zero skew: $d_1 = d_2$ (default choice)

Useful skew: $d_1 - d_2 = \Delta t_2$ (used to maximize clock frequency / timing optimization)

Q? How to get a zero-skew clock network?



Routing

Modern designs could have 100s of millions to billions of nets to be routed

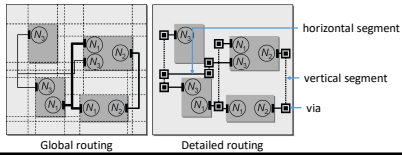
Divide and conquer: global and detailed routing

Global routing

- Coarse grid, and rough routing (which global routing grids to go through)
- Don't worry about exact routing tracks to use or detailed design rules

Detailed routing

- Need to decide the exact routing geometry, wire routing layer, vias, widths, spacing
- Need to meet all design rules

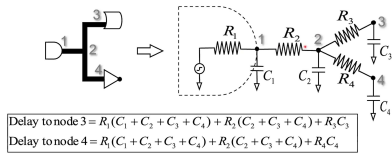


Timing Analysis

We need timing metrics to guide optimization at all stages

Timing model for transistors/gates or interconnects

Accuracy vs fidelity – Elmore delay



Static timing analysis

- Arrival time
- Required time
- Slack = required time – arrival time
- [Incremental computation](#)